Hybrid Search Stategies

3) MMR [Maximal Marginal Relevance]

Query: Langchain GenAI ⇒ VectorStore: ① Langchain GenAI ✓ ✓ ② Langchain Agentic AI ✓ ✓ ③ Lang: GenAI and applicat ✓

What is MMR?

MMR (Maximal Marginal Relevance) is a powerful diversity-aware retrieval technique used in information retrieval and RAG pipelines to balance relevance and novelty when selecting documents.

MMR selects documents that are both:
1. **Relevant to the query** ✓
2. **Diverse from each other (non-redundant)** ✓

} Aim

It prevents the retriever from returning very similar documents that repeat the same content.

$$MMR(d) = \lambda * sim(d,q) - (1-\lambda) \times \max_{s \in S} sim(d,s)$$

1) q = Query
2) Candidate document set D
3) Select Document (s)
4) Similarity function $sim(a,b)$ (Eg: Cosine)

Here $\lambda \in [0,1]$

Tunable parameter

1) Relevance to the q ($\uparrow\lambda$)
2) Diversity (low λ)

Problem

Query: How to use Langchain for RAG

Documents

D1 Langchain enable retrieval with FAISS

D2 Langchain can use Chroma and Pinecone too

D3 Langchain agents can call external tools

MMR

1) Pick Document ⟶ highest similarity (doc, query)

Doc	Similarity Score (cosine Similarity) Sim(doc, Q)
D1	0.95 ✓
D2	0.93
D3	0.80

1) Relevant Document ⟶ D1.

Step 2 : Select Second Document using MMR.

Compare D2 and D3

i) Relevance to query
2) Redundancy with select Doc (D1)

$$MMR(D2) = \lambda * Sim(d2, q) - (1-\lambda) * \max_{s \in S} Sim(doc, s)$$

D1

Sim (doc, S)	Similarity Score
Sim (D1, D2)	0.90 [Redundant]
Sim (D1, D3)	0.30 [Diverse]
Sim (D2, D3)	0.40

$\lambda = 0.7$

Sim (D1, D2)

$$MMR(D2) = 0.7 \times 0.93 + 0.3 * 0.90 = 0.651 - 0.27 = \boxed{0.381}$$

MMR (D3) = 0.7 * 0.80 + 0.3 * Sim (D1, D3)

= 0.7 * 0.80 + 0.3 * 0.30 = 0.56 - 0.09 = **0.47** ↑↑

D1 → Relevance of the Query (Sim (d, q))

D3 → MMR Score

Query

Rank	Document	Reason
1	D1	Highest Relevant
2	D3	But diversity + relevance

[MMR] ←

When to Use MMR [Interviews]

1) In A RAG — To avoid feeding the LLM redundant documents

Eg: langchain

1) FAISS 2) Agents 3) Memory 4) Prompt chain 5) Hybrid Retrieval

→ Result is richer, more useful input or context to LLM.

2) Chatbots : FAQ, SEARCH APP, DOCUMENT BROWSER

3) Retriever Already Return Many Result + Diversity

4) MMR + Hybrid Retrieval → Dense + Sparse

⚖️ When Not to Use MMR

❌ Scenario	🚫 Why You Might Skip MMR
Extremely short context window	You may just want top-1 most relevant
You need **precision only** ✓	Not focused on coverage
Documents are already diverse	No need to enforce diversity
You're already reranking with LLM	Redundancy handled by post-filtering

Query Enhancement

1) Query Expansion Technique

Query Enhancement – Query Expansion Techniques

In a RAG pipeline, the quality of the query sent to the retriever determines how good the retrieved context is — and therefore, how accurate the LLM's final answer will be.

That's where **Query Expansion / Enhancement** comes in.

What is Query Enhancement?

Query enhancement refers to techniques used to improve or reformulate the user query to retrieve better, more relevant documents from the knowledge base.

→ Vector store

It is especially useful when:

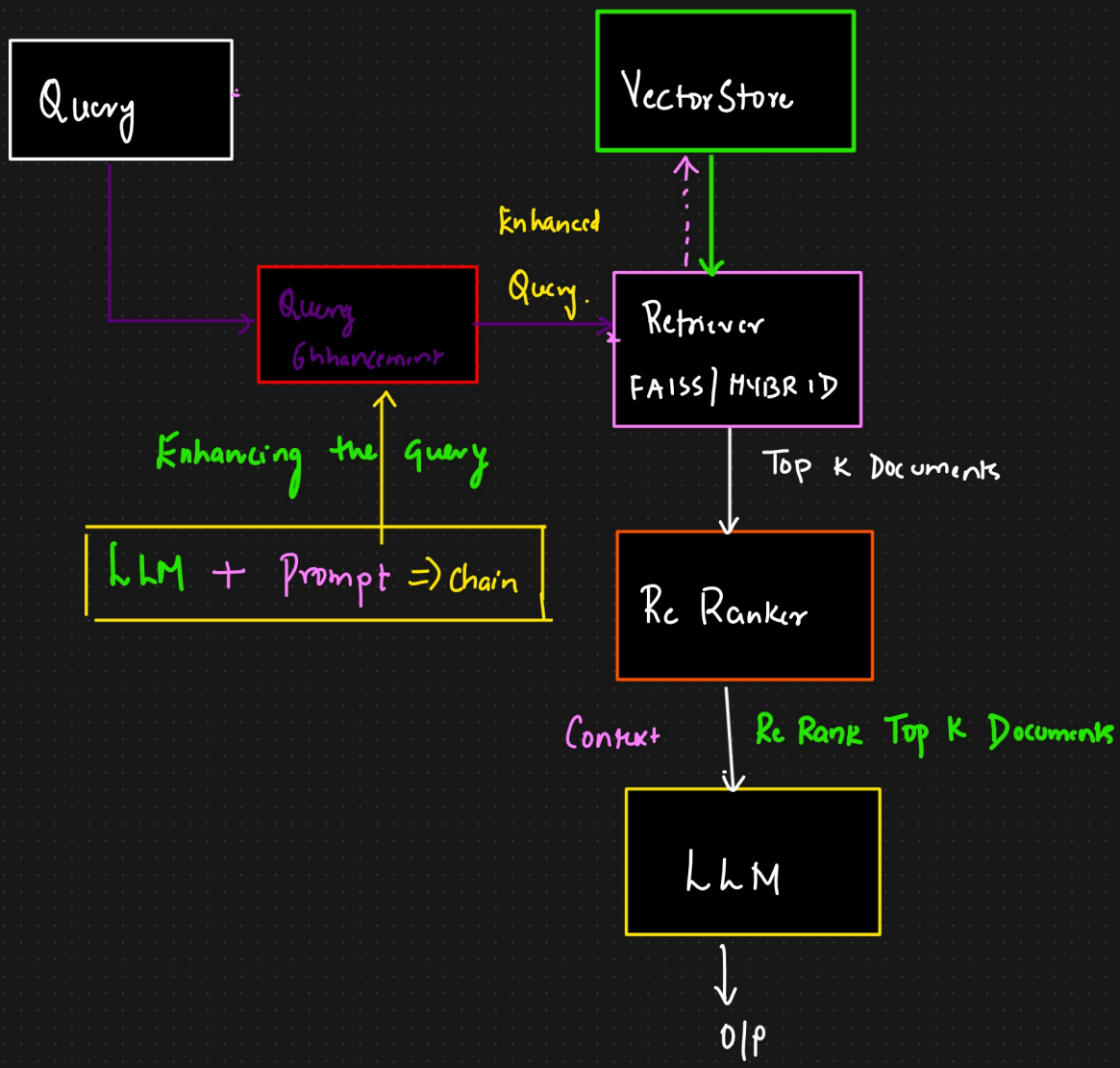
1. **The original query is short, ambiguous, or under-specified**
2. **You want to broaden the scope to catch synonyms, related phrases, or spelling variants**

✅ Why Query Expansion Matters in RAG

Problem (Original Query)	Solution (Enhanced Query)
"LangChain memory"	"LangChain memory modules, conversation memory"
"tools in LLM"	"LangChain tools, APIs, calculator, agent tools"
"retrieval"	"vector retrieval, dense search, BM25, MMR"

Better queries → Better retrieved chunks → Better grounded LLM answers ✅

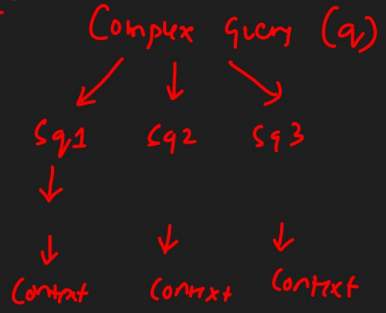
Query Expansion Work



Query Enhancement

🧠 What is Query Decomposition?

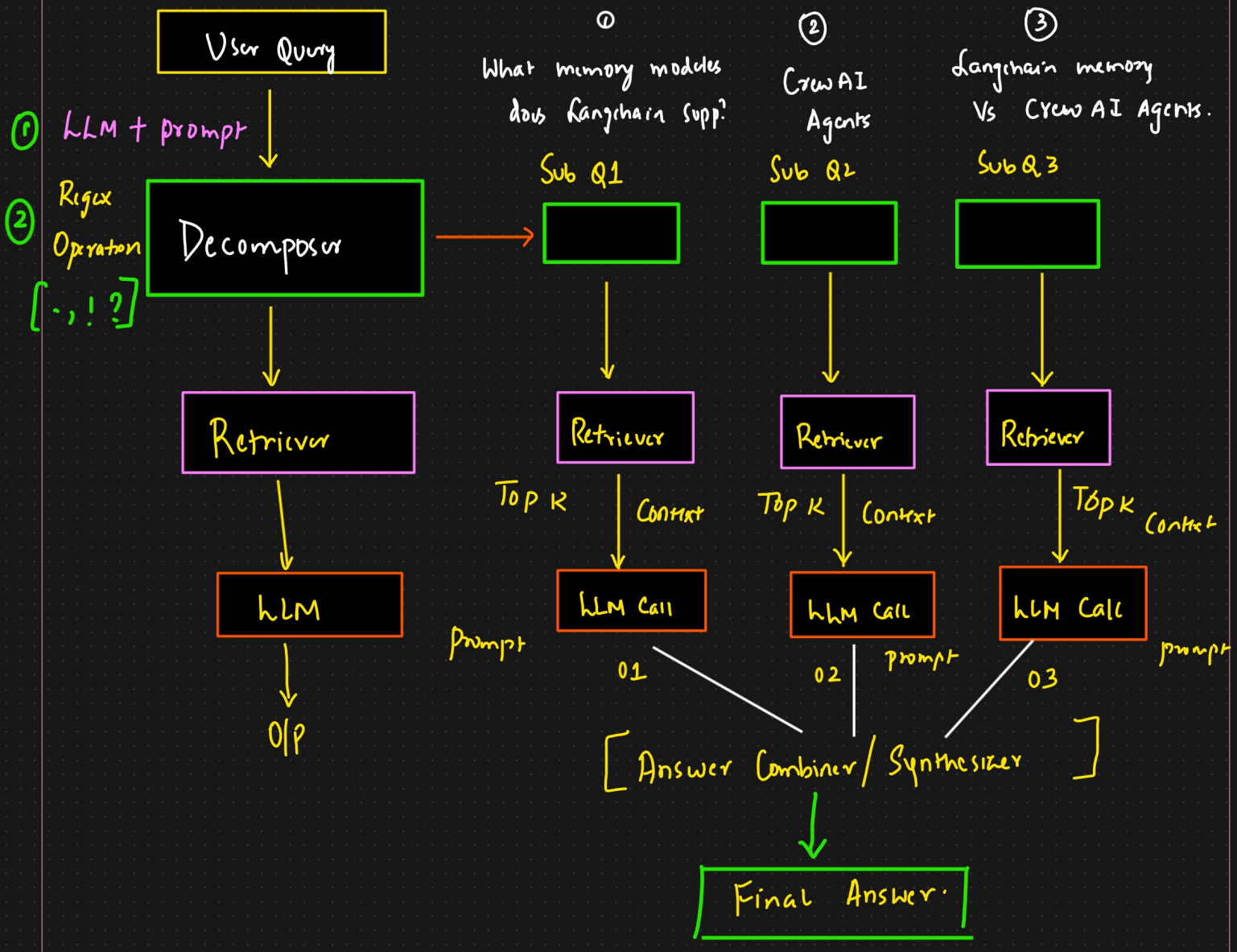
Query decomposition is the process of taking a complex, multi-part question and breaking it into simpler, atomic sub-questions that can each be retrieved and answered individually.



✅ Why Use Query Decomposition?

- Complex queries often involve multiple concepts
- LLMs or retrievers may miss parts of the original question
- It enables multi-hop reasoning (answering in steps)
- Allows parallelism (especially in multi-agent frameworks) ←

Q: What memory modules does Langchain support and how are they different CrewAI



Major Disadvantage

1. LLM calls and Retrieval call.

Query Enhancement

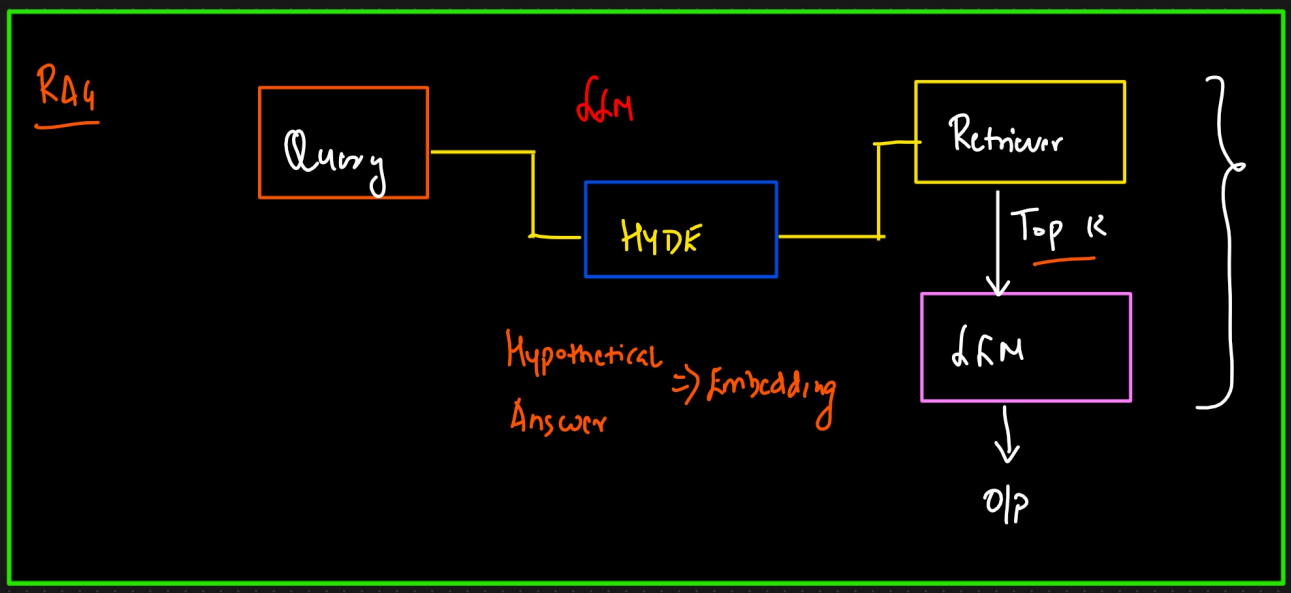
📚 Hypothetical Document Embeddings (HyDE)

🧠 What is HyDe?

HyDE (Hypothetical Document Embeddings) is a retrieval technique where, instead of embedding the user's query directly, you first generate a hypothetical answer (document) to the query using an LLM — and then embed that hypothetical document to search your vector store.

➡️ HyDE bridges the gap between user intent and relevant content, especially when:

1. Queries are short
2. Language mismatch between query and documents
3.You want to retrieve based on answer content, not question words



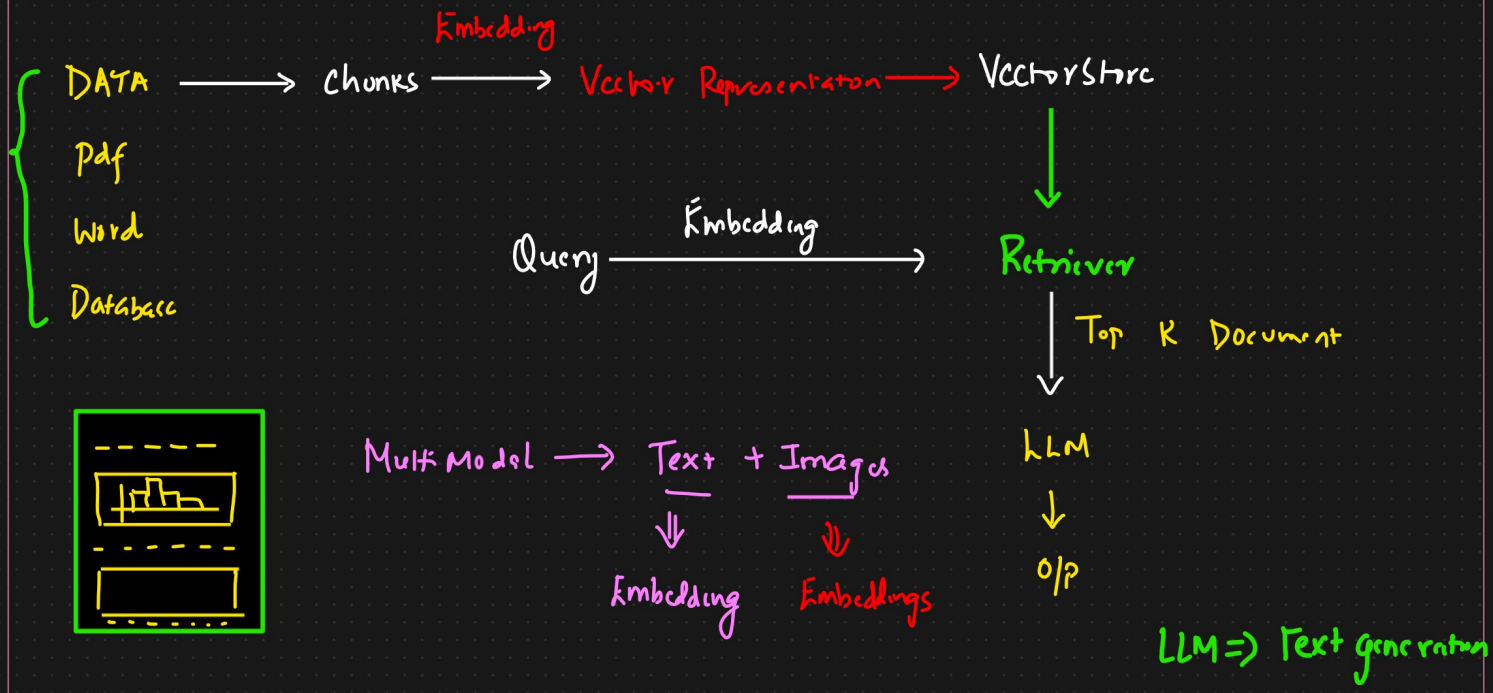
🎯 Why Use HyDE?

Problem	How HyDE Helps
User query and document use different words	Embeds answer-style content instead of question
Query is too vague	LLM-generated answer gives richer semantics
Better grounding needed	Embeds what the answer might look like
Zero-shot retrieval	Works well even without prior training

✅ Benefits of HyDE

Feature	Why It Helps
Semantic intent modeling	Better than literal keyword matching
LLM-aware retrieval	Query is expanded into more context
Generalization	Works well even if document phrasing differs from question
Plug-and-play	No need for retraining; works with OpenAI, Cohere, Hugging Face

MultiModal RAG



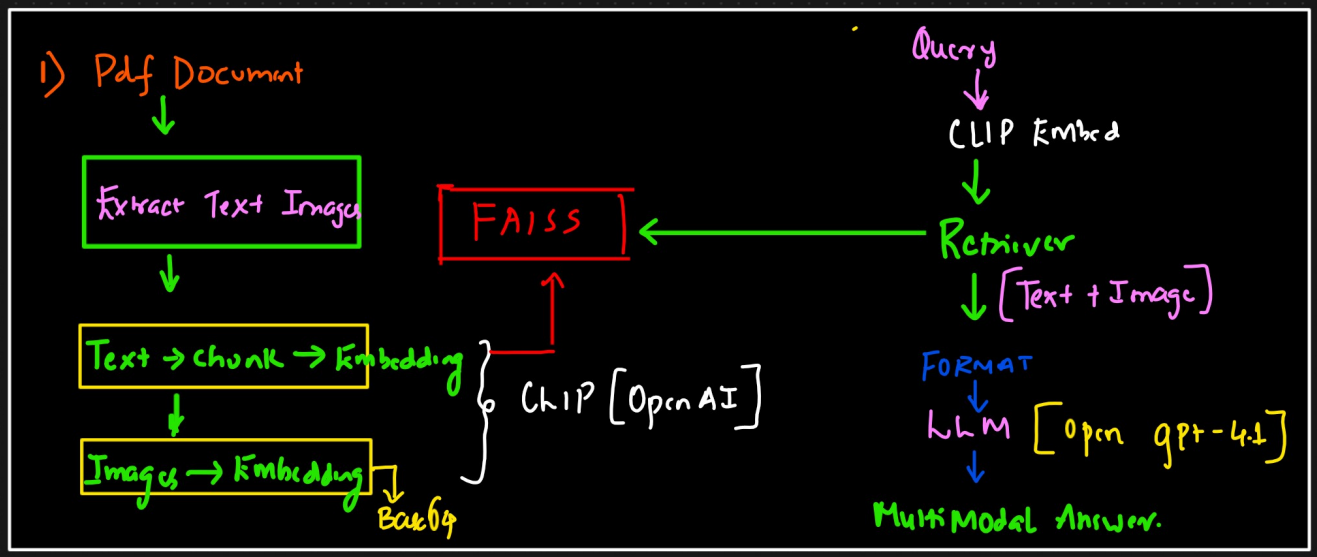
LLM => Text generation

MultiModal LLM => Text + Images

=> OpenAI : gpt-4.1

Google Gemini flash 2.5

Steps : CLIP : Contrastive Language - Image Pre Training.



CLIP : Vision Transformer + Transformer